API_Task(1)

1. What are the fundamental differences between the original ASP.NET and ASP.NET Core?

- Cross-Platform:
  The original ASP.NET worked only on Windows and depended on IIS.
  ASP.NET Core is cross-platform running on Windows, macOS, and Linux (Open Source)
- Performance:
  ASP.NET Core is much faster and designed for high-performance and cloud applications
- Dependency Injection:
  In original ASP.NET, DI was not built-in
  In ASP.NET Core, DI is a first-class feature and used throughout the system

2. What does it mean for an API to be "Stateless"?

Every request to the server is independent

3. Break down the anatomy of an HTTP Request URL?

https://www.example.com:5000/api/data?id=10

Part	Name
`https`	**Scheme**
`www.example.com`	**Host**
`5000`	**Port**
`/api/data`	**Path**
`?id=10`	**Query String**

4. What are the primary HTTP Methods (Verbs) and their standard Methods ?

Methods	Usage
Get	Get data from API
Post	Create new record
Put	Update existing records
Delete	Delete records

5. What is the role of Program.cs in an ASP.NET Core application?

- Build and configure the app
- Define routing
- Run the application

6. Why is Swagger/OpenAPI typically enabled only in the "Development" environment?

- Security: Shows all API endpoints, can be risky in Production
- Performance: Generating the documentation adds unnecessary overhead in a production environment where users only need the API to function, not a UI to browse it

7. What is the core concept of "Dependency Injection" (DI)?

Dependency Injection is a design pattern where an object receives the other objects it depends on, rather than creating them itself

8. Explain the three Service Lifetimes in ASP.NET Core DI?

Transient: new instance every time. Scoped: one per request. Singleton: one for the lifetime of the app.

9. Why is it a "Best Practice" to register services against an Interface rather than a concrete Class?

1. Loose Coupling: Your code depends on an interface, not a specific class

2. Easier Testing
3. Better Maintainability & Flexibility: Makes your code more reusable and cleaner

10. What are the "Launch Profiles" found in launchSettings.json ?

define how your app runs during development

Application URL / Port , Environment

11. How does JSON facilitate data exchange in APIs?

- JSON (JavaScript Object Notation) is a lightweight format for sending data between a client and a server
- Easy to read and write: Both humans and machines can understand it
- Language-independent: Supported by almost every programming language

Question 1 - What is Dependency?

```
public class OrderService
{
        public void CreateOrder(Order order)
        {
            // Save to database
            var connection = new SqlConnection("connection_string");
            // ... database logic
            // Send email
            var emailSender = new EmailSender();
            emailSender.Send("Order created!");
            // Log
            var logger = new FileLogger();
            logger.Log("Order created");
        }
}
```

ال `OrderService` بتعرف الـ dependencies بتاعتها جواها ودا بيسبب:

Tight Coupling Problem

- Difficult to test
- Low flexibility
- Poor maintainability and scalability
- Lack of abstraction – no interfaces are used

Question 2 - Tight Coupling Problem
What's the difference between A and B? Which one is better?

```
// Scenario A
public class UserService
{
        private EmailService _emailService = new EmailService();
        public void RegisterUser(User user)
        {
                // Save user...
                _emailService.SendWelcomeEmail(user.Email);
        }
}
// Scenario B
public class UserService
{
```

```
        private readonly IEmailService _emailService;
        public UserService(IEmailService emailService)
        {
                _emailService = emailService;
        }

        public void RegisterUser(User user)
        {
                // Save user...
                _emailService.SendWelcomeEmail(user.Email);
        }
}
```

// Scenario A

ال `UserService` بتعمل الdependencies بتاعتها جواها ودا بيسبب Tight Coupling Problem وصعوبه في test مفيش flexibility و مشاكل في ال Maintainability

// Scenario B

هنا ال `UserService` بدل م تعمل ال dependencies بتاعتها جواها بتستلمهم من برا جاهزين عن طريق الConstructor (تعتمد علي Abstraction), اقدر اغير implementation اي method من غير م `UserService` تتغير, Loose Coupling وبالتالي سهوله في test و Maintainability و Scalability

→ Scenario B is the better

Question 3 - Constructor Injection
What happens when a request is received?

```
public interface IRepository
{
        void Save(object data);
}

public class SqlRepository : IRepository
{
        public void Save(object data)
        {
                Console.WriteLine("Saving to SQL Database");
        }
}
public class ProductService
{
        private readonly IRepository _repository;

        // Constructor Injection
        public ProductService(IRepository repository)
        {
                _repository = repository;
        }
        public void AddProduct(string product)
        {
                Console.WriteLine( "Adding product: {product}");
                _repository.Save(product);
        }
}
// in ASP.NET Core Startup/Program.cs
builder.Services.AddScoped<IRepository, SqlRepository>();
builder.Services.AddScoped<ProductService>();
// in Controller
public class ProductController : ControllerBase
{
```

```
        private readonly ProductService _productService;
        public ProductController(ProductService productService)
        {
                _productService = productService;
        }
        [HttpPost]
        public IActionResult Add(string name)
        {
                _productService.AddProduct(name);
                return Ok();
        }
}
```

1. ال request بيتوجه للـ `ProductController`
2. ASP.NET Core بيستخدم Dependency Injection علشان ينشئ الـ Controller
3. وال Controller محتاج `ProductService` في ال Constructor، فـ DI Container يبدأ يبحث عنه
4. وأثناء إنشاء `ProductService` بيلاقي إنه محتاج `IRepository`
5. فـ ASP.NET Core يربطه بال `SqlRepository`
6. وبيتم حقن كل الـ dependencies عن طريق ال Constructor
7. بعد كده بيتنفذ ال `Add` method
8. ال `ProductService` بينادي `_repository.Save()`
9. وال `SqlRepository` هو اللي بينفذ عملية الحفظ `Save`
10. وفي الآخر ال API بيرجع `Ok` Response

Question 4 - DI Container Registration
What is the output for each registration?

```
// Registration A
builder.Services.AddTransient<IEmailService, EmailService>();

//new instance every time ->email1, email2 not Same Instance
//AreSameInstance = false


// Registration B
builder.Services.AddScoped<IEmailService, EmailService>();

//one per request ->email1, email2 in same request -> Same Instance
//AreSameInstance = true


// Registration C
builder.Services.AddSingleton<IEmailService, EmailService>();

//one for the lifetime of the app ->email1, email2 Same Instance
//AreSameInstance = true


// Controller
public class HomeController : ControllerBase
{
        private readonly IEmailService _email1;
```

```
        private readonly IEmailService _email2;
        public HomeController(
        IEmailService email1,
        IEmailService email2)
        {
                _email1 = email1;
                _email2 = email2;
        }

        [HttpGet]
        public IActionResult Test()
        {
                var same = Object.ReferenceEquals(_email1, _email2);
                return Ok(new { AreSameInstance = same });
        }
}
```

Question 5 - Multiple Registrations
Which implementation will Controller A receive?
How many services will be injected into Controller B?

```
// Registration
builder.Services.AddScoped<IEmailService, SmtpEmailService>();
builder.Services.AddScoped<IEmailService, SendGridEmailService>();
builder.Services.AddScoped<IEmailService, MailgunEmailService>();
// Controller A
public class ControllerA
{
        private readonly IEmailService _email;
        public ControllerA(IEmailService email)
        {
                _email = email; // Which one?
        }
}

//one object one service (اخر Registration بي override علي الباقي)
//Which one? MailgunEmailService



// Controller B
public class ControllerB
{
        private readonly IEnumerable<IEmailService> _emails;

        public ControllerB(IEnumerable<IEmailService> emails)
        {
                _emails = emails; // How many?
        }
}

//IEnumerable->list of services
// How many? 3 services
```